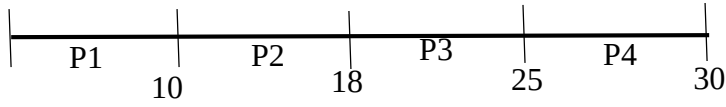


CPU Zamanlama Algoritmaları Örnek Soru Üzerinde Karşılaştırma

Örnek 1: Aşağıda verilen soruyu FIFO algoritmasına göre çözelim kesmeyen (nonpreemptive) yapıya göre çözelim.

	Gelme Zamanı	CPU Burst Zamanı
P1	0	10
P2	0	8
P3	2	7
P4	2	5

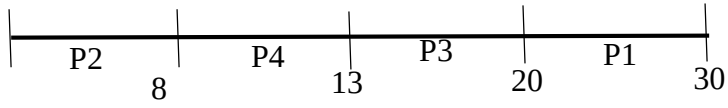


	Bekleme Zamanı	Cevap Zamanı
P1	0	10
P2	10	8
P3	16	7
P4	23	5
Toplam	49	30
Ortalama	12.25	7.5

Cevap zamanında ilk ne zaman cevap verilmiş ise onunla ilgilenilir. Diğer duruma bakılmaz.

Örnek 2: Aşağıda verilen soruyu Shortest Job First algoritmasına göre çözelim kesmeyen (nonpreemptive) yapıya göre çözelim.

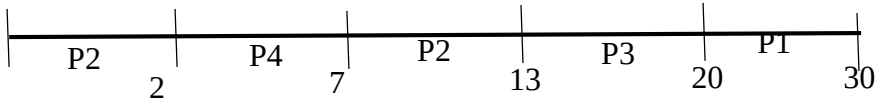
	Gelme Zamanı	CPU Burst Zamanı
P1	0	10
P2	0	8
P3	2	7
P4	2	5



	Bekleme Zamanı	Cevap Zamanı
P1	20	10
P2	0	8
P3	11	7
P4	6	5
Toplam	37	30
Ortalama	9,25	7.5

Örnek 3: Aşağıda verilen soruyu Shortest Job First algoritmasına göre çözelim kesen (preemptive) yapıya göre çözelim.

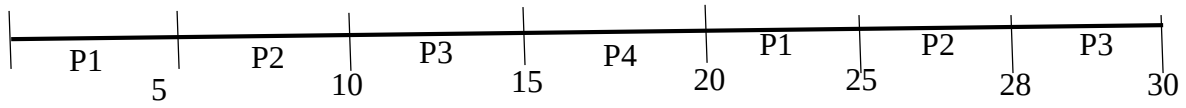
	Gelme Zamanı	CPU Burst Zamanı
P1	0	10
P2	0	8
P3	2	7
P4	2	5



	Bekleme Zamanı	Cevap Zamanı
P1	20	20
P2	5	0
P3	11	11
P4	0	0
Toplam	36	31
Ortalama	9	7.75

Örnek 4: Aşağıda verilen soruyu Round Robin algoritmasına göre çözelim Time Quantum=5 tir.

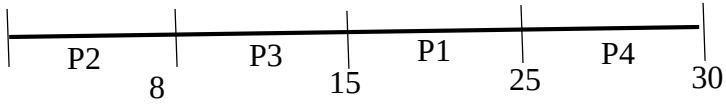
	Gelme Zamanı	CPU Burst Zamanı
P1	0	10
P2	0	8
P3	2	7
P4	2	5



	Bekleme Zamanı	Cevap Zamanı
P1	(0+15)	0
P2	(5+15)	5
P3	(8+13)	8
P4	13	13
Toplam	69	26
Ortalama	17.25	6.5

Örnek 5: Aşağıda verilen soruyu Priority Zamanlama göre çözünüz. Priority yüksek olan önceliklidir kabulü ile çözüm yapılmalıdır.

	Gelme Zamanı	CPU Burst Zamanı	Priority Zamanı
P1	0	10	1
P2	0	8	3
P3	2	7	3
P4	2	5	1



	Bekleme Zamanı	Cevap Zamanı
P1	15	15
P2	0	0
P3	6	6
P4	23	23
Toplam	44	44
Ortalama	11	11

Bölüm 6: İşlem Senkronizasyonu (1)





Arkaplan Bilgisi

- Paylaşılan veriye aynı anda erişim veride tutarsızlıklara neden olabilir
- Verinin tutarlılığını korumak, veriye ortak erişen işlemlerin veriye erişimlerini sıraya sokan bir mekanizmayı gerektirir
- Üretici-tüketici probleminde tüm tampon belleği dolduracak bir çözüm sunmak istediğimizi varsayalım
- Bunu dolu bellek hücrelerini saymakta kullanacağımız **count** adında bir tamsayı sayaç ile sağlayabiliriz
 - Başlangıçta count sıfıra eşitlenecektir
 - Üretici yeni bir tampon bellek hücrecini doldurduğunda count bir artacaktır
 - Tüketici bir tampon bellek hücresindeki veriyi tükettiğinde ise count bir azalacaktır





Üretici

```
while (count == BUFFER.SIZE)
    ; // do nothing

// add an item to the buffer
buffer[in] = item;
in = (in + 1) % BUFFER.SIZE;
++count;
```





Tüketici

```
while (count == 0)
    ; // do nothing

// remove an item from the
buffer item = buffer[out];
out = (out + 1) % BUFFER.SIZE;
--count;
```





Yarışma Durumu - Örnek

- `count++` şu şekilde gerçekleştirilebilir

```
register1 = count  
register1 = register1 + 1  
count = register1
```

- `count--` şu şekilde gerçekleştirilebilir

```
register2 = count  
register2 = register2 - 1  
count = register2
```

- “count = 5” iken aşağıdaki işlemlerin gerçekleştiğini varsayın:

T0: üreticinin çalıştırdığı kod: `register1 = count` {register1 = 5}

T1: üreticinin çalıştırdığı kod: `register1 = register1 + 1` {register1 = 6}

T2: tüketicinin çalıştırdığı kod: `register2 = count` {register2 = 5}

T3: tüketicinin çalıştırdığı kod: `register2 = register2 - 1` {register2 = 4}

T4: üreticinin çalıştırdığı kod: `count = register1` {count = 6}

T5: tüketicinin çalıştırdığı kod: `count = register2` {count = 4}





Yarışma Durumu - Tanım

- Pek çok işlemin **aynı anda** bir veriye erişmek ve onu değiştirmek istediği durumlarda işlemlerin çalışması sonucu elde edilen sonucun işlemlerin veriye eriştiği sıraya bağlı olduğu durumlara yarışma durumları (race condition) denir
- Yarışma durumunda tutarlı sonuç elde etmek için, count değişkenine aynı anda sadece bir işlemin erişmesini sağlamalıyız. Bu da işlemlerin senkronizasyonu ile mümkündür
- Yarışma durumları işletim sistemlerinde çok karşılaşılan bir durumdur
- Bunun nedeni kaynakların (örn: hafıza, I/O cihazları) pek çok bileşen tarafından paylaşılıyor olmasıdır
- Çok çekirdekli işlemcilerin ve iş parçacıklarının kullanımı da durumu giderek daha karmaşık hale getirmektedir





Kritik-kısım Problemi

- n tane işlemin olduğu bir sistem düşünelim
- Her bir işlemin bir kısım kodunun aşağıdaki işlemlerden birini yapan bir kritik-kısıma sahip olduğunu düşünün
 - Ortak bir değişkenin değerini değiştiren
 - Ortak bir tabloyu güncelleyen
 - Ortak kullanılan bir dosyayı güncelleyen
- Böyle bir sistemin tutarlı sonuç üretmesi için kritik-kısıma erişimi, aynı anda bir işlemin erişebileceği şekilde sınırlandırmalıyız





Kritik-kısım Problemine Çözüm

Kritik-kısım problemine önerilen çözüm aşağıdaki kriterleri sağlamalıdır:

1. Karşılıklı Dışlama (mutual exclusion) – Eğer işlem P_i kritik kısımda çalışıyorsa, diğer işlemler kritik kısımda çalışamaz
2. İlerleme (progress) – Eğer kritik kısımda çalışan bir işlem yoksa ve bazı işlemler kritik kısımda çalışmak istiyorsa, bu işlemlerden birini seçip çalıştırmak sonsuza kadar ertelenmemelidir
3. Sınırlı Bekleme (bounded waiting) - Kritik kısma girmek isteyen bir işlemin bekleme süresi sınırlandırılmalıdır. O işlem beklerken, diğer işlemlerden en fazla belirlenen sayıda işlem kritik kısma girmelidir. Ardından bekleyen işlemin kritik kısma girmesine izin verilmelidir.
 - Her bir işlemin sıfır dışında bir hızda çalıştığı varsayılmaktadır
 - Bu N işlemin bağıl hızları hakkında herhangi bir varsayımımız yoktur





Tipik Bir İşlemin Yapısı

```
while (true) {  
    entry section  
    critical section  
    exit section  
    remainder section  
}
```





Peterson'un Çözümü

- İki işlem çözümü
- LOAD ve STORE koutlarının atomik olduğunu varsayın
- Bu iki işlem iki değişken paylaşır:
 - int **turn**;
 - boolean **flag[2]**
- **turn** değişkeni kritik kısma giriş sırasının kimde olduğunu belirtiyor
- **flag** dizisi bir işlemin kritik kısma girişe hazır olup olmadığını belirtiyor.
flag[i] = true **P_i** işleminin hazır olduğunu gösteriyor





P_i İşlemi için Algoritma

```
while (true) {
```

```
    flag[i] = true;
```

```
    turn = j;
```

```
    while (flag[j] && turn == j);
```

```
    critical section
```

```
    flag[i] = false;
```

```
    remainder section
```

```
}
```

Dikkat: $j = 1 - i$





Kilitleri Kullanarak Kritik-kısım Problemi Çözümü

- Yazılım tabanlı çözümlerin (Peterson'un algoritması gibi) modern bilgisayar mimarilerinde çalışmasının garantisi yoktur.
- Genel olarak kritik-kısım problemini çözmek için küçük bir araca ihtiyacımız olduğunu söyleyebiliriz: kilit (lock)

```
while (true) {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
}
```





Senkronizasyon Donanımı

- Kritik-kısım problemi için pek çok sistem donanım desteği sunmaktadır
- Tek işlemcili sistemler – geçici olarak kesintileri (interrupts) iptal edebilirler
 - O an çalışan kod bölünmeden çalışmaya devam edebilir
 - Genel olarak çok işlemcili bilgisayarlarda verimli değildir - işlemciler arasında mesajlaşma gerektirir
 - ▶ Bu özelliği kullanan işletim sistemleri ölçeklenebilir değildir
- Modern makinalar özel atomik donanım komutları sağlarlar
 - ▶ Atomik = kesilmeyen (non-interruptable)
 - Hafıza hücresinin değerini değiştirir veya test eder
 - Veya iki hafıza hücresinin değiş tokuş eder





Semafor (Semaphore)

- Meşgul bekleme (busy waiting) gerektirmeyen senkronizasyon aracı
- Semafor S – tamsayı değişken
- S üzerinde iki standard işlem : **acquire()** ve **release()**
 - Orijinal olarak **P()** ve **V()**
- Daha az karmaşık
- Sadece iki atomik işlem ile erişilebiliyor

```
acquire() {  
    while value <= 0  
        ; // no-op  
    value--;  
}  
  
release() {  
    value++;  
}
```





Semafor Genel Senkronizasyon Aracı

- **Sayaç semaforu** (counting semaphore) – tamsayı değeri sınırsız bir değer aralığına sahiptir
- **İkili semafor** (binary semaphore) – tamsayı değeri sadece 0 ya da 1 değerlerini alabilir; gerçekleştirimi daha basit olabilir
 - **mutex lock** olarak da bilinir:

```
Semaphore sem = new Semaphore(1);  
  
sem.acquire();  
  
    // critical section  
  
sem.release();  
  
    // remainder section
```



SEMAFOR

Semaforlar, süreç senkronizasyonunda ve çoklu iş parçacığında çok kullanışlıdır.

Linux sistemlerinde POSIX semafor kütüphanesi bulunmaktadır. Aslında semaforun temel kodu örnekte gösterildiği gibi basittir. Ancak bu kod doğrudan yazılamaz, çünkü işlevlerin atomik olması gerekir ve doğrudan kod yazmak, işlev tamamlanmadan bir bağlam değişikliğine yol açar ve bir karışıklığa neden olur. Linux'taki POSIX sistemi, kendi yerleşik semafor kitaplığını sunar.

1. Include semaphore.h
2. Compile the code by linking with -lpthread -lrt

Bir semaforu kilitlemek veya beklemek için **sem_wait** işlevini kullanabiliriz:

```
int sem_wait(sem_t *sem);
```

3. Bir semaforu serbest bırakmak veya işaret etmek için **sem_post** işlevini kullanırız.:

```
int sem_post(sem_t *sem);
```

4. Bir semafor, **sem_init**(işlemler veya iş parçacıkları için) kullanılarak başlatılır.)
or **sem_open** (for IPC).

```
sem_init(sem_t *sem, int pshared, unsigned int value);
```

- **sem** : Başlatılacak semaforu belirtir.
- **pshared** : Bu argüman, yeni başlatılan semaforun süreçler arasında mı yoksa iş parçacıkları arasında mı paylaşılacağını belirtir. Sıfır olmayan bir değer, semaforun süreçler arasında paylaşıldığı ve sıfır değeri, iş parçacıkları arasında paylaşıldığı anlamına gelir..
- **value** : Yeni başlatılan semafora atanacak değeri belirtir.
- Bir semaforu yok etmek için **sem_destroy** kullanabiliriz. **sem_destroy(sem_t *mutex);**

Bir semafor bildirmek için veri türü **sem_t**'dir.

```
// Semaforların çalışmasını gösteren C programı
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t mutex;

void* thread(void* arg)
{
    //wait
    sem_wait(&mutex);
    printf("\nGiris..\n");

    //critical section
    sleep(4);

    //signal
    printf("\nAz önce çıkış...\n");
    sem_post(&mutex);
}

int main()
{
    sem_init(&mutex, 0, 1);
    pthread_t t1,t2;
    pthread_create(&t1,NULL,thread,NULL);
    sleep(2);
    pthread_create(&t2,NULL,thread,NULL);
    pthread_join(t1,NULL);
    pthread_join(t2,NULL);
    sem_destroy(&mutex);
    return 0;
}
```

Kod Çıktısı:

Giris..

Az önce çıkış...

Giris..

Az önce çıkış...

Açıklama: Biri ilkinden 2 saniye sonra olmak üzere 2 iş parçacığı oluşturuluyor. Ancak ilk iş parçacığı kilidi aldıktan sonra 4 saniye uyuyacaktır.

Böylece ikinci thread çağrıldıktan hemen sonra girmeyecek, çağrıldıktan $4 - 2 = 2$ sn sonra girecektir.